

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I Geferlin Boffy R (192571080) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Online Shopping Platform – Normalize Order to 3NF

Given Relation

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Assumptions

- Each OrderID identifies one order.
- Each CustID identifies one customer.
- Each ProdID identifies one product.
- An order can contain multiple products.
- A product occurs at most once in a particular order.
- Price is the standard price of the product.

Step 1: Identify Functional Dependencies

The functional dependencies are:

1. **OrderID $\rightarrow$ CustID**
2. **CustID $\rightarrow$ CustName**
3. **ProdID $\rightarrow$ ProdName, Price**
4. **(OrderID, ProdID) $\rightarrow$ Qty**

Therefore:

OrderID $\rightarrow$ CustID $\rightarrow$ CustName

is a transitive dependency.

The composite key is:

(OrderID, ProdID)

because an order may contain multiple products.

Step 2: Identify Partial Dependencies

Since the candidate key is **(OrderID, ProdID)**:

- OrderID $\rightarrow$ CustID
- ProdID $\rightarrow$ ProdName, Price

These attributes depend only on part of the composite key.

Therefore, partial dependencies exist, so the relation is not in 2NF.

Step 3: Decompose into 2NF

Create:

ORDER_HEADER(OrderID, CustID)

PRODUCT(ProdID, ProdName, Price)

ORDER_ITEM(OrderID, ProdID, Qty)

Step 4: Remove Transitive Dependency for 3NF

The dependency:

OrderID → CustID → CustName

is transitive.

Therefore, create a separate customer relation:

CUSTOMER(CustID, CustName)

Final 3NF Schema

CUSTOMER(CustID, CustName)

Primary Key: **CustID**

PRODUCT(ProdID, ProdName, Price)

Primary Key: **ProdID**

ORDER_HEADER(OrderID, CustID)

Primary Key: **OrderID**

Foreign Key: **CustID**

ORDER_ITEM(OrderID, ProdID, Qty)

Primary Key: **(OrderID, ProdID)**

Justification

The decomposition removes partial and transitive dependencies. Customer, product, order, and order-item information are stored separately, reducing redundancy and preventing insertion, deletion, and update anomalies.

Conclusion

The original ORDER relation is normalized to **3NF** through the relations CUSTOMER, PRODUCT, ORDER_HEADER, and ORDER_ITEM.

2. Hospital Database – Normalize Patient Relation to BCNF

Given Relation

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Assumptions

- Each PID identifies one patient.
- Each doctor has a unique DocID.
- Each ward has a unique WardNo.
- A patient is assigned to one doctor and one ward.

Step 1: Functional Dependencies

The functional dependencies are:

1. **$PID \rightarrow PName, DocID, WardNo$**
2. **$DocID \rightarrow DocName, Specialization$**
3. **$WardNo \rightarrow WardName$**

Therefore:

$PID \rightarrow DocID \rightarrow DocName, Specialization$

and:

$PID \rightarrow WardNo \rightarrow WardName$

are transitive dependencies.

Step 2: Candidate Key

Calculate PID closure:

$PID^+ = \{PID\}$

Using $PID \rightarrow PName, DocID, WardNo$:

$PID^+ = \{PID, PName, DocID, WardNo\}$

Using $DocID \rightarrow DocName, Specialization$:

$PID^+ = \{PID, PName, DocID, DocName, Specialization, WardNo\}$

Using $WardNo \rightarrow WardName$:

$PID^+ = \{PID, PName, DocID, DocName, Specialization, WardNo, WardName\}$

Therefore:

Candidate Key = PID

Step 3: Identify Anomalies

Update Anomaly

If a doctor's specialization changes, the change must be made in multiple patient records.

Insertion Anomaly

A new doctor cannot be stored properly unless a patient is assigned to that doctor.

Deletion Anomaly

Deleting the last patient assigned to a doctor may also remove the only stored information about that doctor.

Step 4: BCNF Decomposition

Create:

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

Step 5: Verify BCNF

In PATIENT:

PID $\rightarrow$ PName, DocID, WardNo

PID is the key.

In DOCTOR:

DocID $\rightarrow$ DocName, Specialization

DocID is the key.

In WARD:

WardNo $\rightarrow$ WardName

WardNo is the key.

Therefore, every determinant is a superkey in its respective relation.

Conclusion

The decomposition:

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

is in **BCNF** and eliminates the major insertion, deletion, and update anomalies.

3. University Enrollment – Analyze FDs and Apply 2NF

Given Relation

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

Assumptions

- SID uniquely identifies a student.
- CourseID uniquely identifies a course.
- A student can enroll in multiple courses.
- A course can have multiple students.
- Grade depends on both student and course.

Step 1: Identify Functional Dependencies

1. **SID $\rightarrow$ SName**
2. **CourseID $\rightarrow$ CourseName, Faculty**
3. **(SID, CourseID) $\rightarrow$ Grade**

Step 2: Determine Candidate Key

Calculate:

(SID, CourseID)+

Starting with:

{SID, CourseID}

Using SID $\rightarrow$ SName:

{SID, CourseID, SName}

Using CourseID $\rightarrow$ CourseName, Faculty:

{SID, CourseID, SName, CourseName, Faculty}

Using (SID, CourseID) $\rightarrow$ Grade:

{SID, CourseID, SName, CourseName, Faculty, Grade}

Therefore:

Candidate Key = (SID, CourseID)

Step 3: Identify Partial Dependencies

The candidate key is composite.

However:

SID $\rightarrow$ SName

SName depends only on SID.

Also:

CourseID $\rightarrow$ CourseName, Faculty

CourseName and Faculty depend only on CourseID.

Therefore, these are partial dependencies.

Hence, the relation is not in 2NF.

Step 4: 2NF Decomposition

Create:

STUDENT(SID, SName)

Primary Key: **SID**

COURSE(CourseID, CourseName, Faculty)

Primary Key: **CourseID**

ENROLLMENT(SID, CourseID, Grade)

Primary Key: **(SID, CourseID)**

Conclusion

The decomposition removes partial dependencies because student details depend only on SID, course details depend only on CourseID, and Grade depends on the complete composite key.

Therefore, the resulting relations are in 2NF.

4. Airline Booking – Normalize to BCNF

Given Relation

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assumptions

- A PassID uniquely identifies a passenger.
- A FlightNo identifies a flight route.
- A flight operates on multiple dates.
- A seat is unique within a particular flight on a particular date.
- A passenger can have one seat on a particular flight/date.

Step 1: Functional Dependencies

1. **PassID $\rightarrow$ PassName**

2. **FlightNo** $\rightarrow$ **DeptCity, ArrCity**
3. **(FlightNo, Date, PassID)** $\rightarrow$ **Seat**
4. **(FlightNo, Date, Seat)** $\rightarrow$ **PassID**

Step 2: Candidate Keys

The combination:

(FlightNo, Date, PassID)

determines all attributes.

Therefore, it is a candidate key.

Also:

(FlightNo, Date, Seat)

determines PassID, and PassID determines PassName.

Therefore, it also determines all attributes.

Hence, candidate keys are:

{FlightNo, Date, PassID}

and

{FlightNo, Date, Seat}

Step 3: Identify BCNF Violations

Consider:

PassID $\rightarrow$ **PassName**

PassID is not a superkey of the original relation.

Therefore, this violates BCNF.

Consider:

FlightNo $\rightarrow$ **DeptCity, ArrCity**

FlightNo is also not a superkey of the original relation because the same flight can operate on different dates and have multiple passengers.

Therefore, this also violates BCNF.

Step 4: BCNF Decomposition

Create:

PASSENGER(PassID, PassName)

FLIGHT(FlightNo, DeptCity, ArrCity)

BOOKING(FlightNo, Date, PassID, Seat)

Step 5: Verify BCNF

In PASSENGER:

PassID → PassName

PassID is the key.

In FLIGHT:

FlightNo → DeptCity, ArrCity

FlightNo is the key.

In BOOKING:

(FlightNo, Date, PassID) → Seat

and:

(FlightNo, Date, Seat) → PassID

Both determinants are candidate keys.

Therefore, all relations satisfy BCNF.

Conclusion

The airline booking schema is normalized to **BCNF** by separating passenger, flight, and booking information.

5. HR System – Functional and Transitive Dependencies and 3NF

Representative HR Relation

Since the question does not specify the exact attributes, consider the following standard multinational HR schema:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, CountryID, CountryName)

Assumptions

- EmpID uniquely identifies an employee.
- DeptID uniquely identifies a department.
- CountryID uniquely identifies a country.
- Each department has one department name and manager.

- Each country has one country name.

Step 1: Functional Dependencies

1. **$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{ManagerID}, \text{CountryID}$**
2. **$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$**
3. **$\text{CountryID} \rightarrow \text{CountryName}$**

Step 2: Identify Transitive Dependencies

From:

$\text{EmpID} \rightarrow \text{DeptID}$

and:

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

we get:

$\text{EmpID} \rightarrow \text{DeptName}, \text{ManagerID}$

This is a transitive dependency.

Similarly:

$\text{EmpID} \rightarrow \text{CountryID}$

and:

$\text{CountryID} \rightarrow \text{CountryName}$

give:

$\text{EmpID} \rightarrow \text{CountryName}$

This is also a transitive dependency.

Step 3: Candidate Key

$\text{EmpID}^+ = \{\text{EmpID}, \text{EmpName}, \text{DeptID}, \text{DeptName}, \text{ManagerID}, \text{CountryID}, \text{CountryName}\}$

Therefore:

Candidate Key = EmpID

Step 4: 3NF Decomposition

Create:

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID, CountryID)

DEPARTMENT(DeptID, DeptName, ManagerID)

COUNTRY(CountryID, CountryName)

Step 5: Verify 3NF

In EMPLOYEE:

EmpID $\rightarrow$ EmpName, DeptID, ManagerID, CountryID

EmpID is the key.

In DEPARTMENT:

DeptID $\rightarrow$ DeptName, ManagerID

DeptID is the key.

In COUNTRY:

CountryID $\rightarrow$ CountryName

CountryID is the key.

Conclusion

The transitive dependencies are removed by separating department and country information. The resulting relations are in **3NF**, with reduced redundancy and improved data consistency.

6. Library – Identify MVDs and Decompose to 4NF

Given Relation

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Important Assumption

A member can borrow multiple books, and the member's collection of books is treated as an independent multivalued fact. Book information is determined by BookID.

Step 1: Functional Dependencies

1. **MemberID $\rightarrow$ MemberName**
2. **BookID $\rightarrow$ Title, Author, Genre**
3. **(MemberID, BookID) $\rightarrow$ BorrowDate**

Step 2: Identify the Multivalued Dependency

If the system treats the set of books associated with a member as an independent multivalued fact, we have:

MemberID $\twoheadrightarrow$ BookID

The MVD causes repeated combinations when independent multivalued facts are stored together.

Step 3: Identify Redundancy

For example, if a member has several books, the member's name is repeatedly stored for every book.

Book details are also repeated whenever the same book is borrowed or associated with different members.

Step 4: Decompose

Create:

MEMBER(MemberID, MemberName)

BOOK(BookID, Title, Author, Genre)

BORROWING(MemberID, BookID, BorrowDate)

Step 5: Verify 4NF

MEMBER:

MemberID → MemberName

MemberID is the key.

BOOK:

BookID → Title, Author, Genre

BookID is the key.

BORROWING:

(MemberID, BookID) → BorrowDate

The complete borrowing combination determines the borrowing date.

Thus, the independent facts are separated and the non-trivial MVD is removed from the larger relation.

Conclusion

The decomposition into MEMBER, BOOK, and BORROWING removes the multivalued redundancy and produces a **4NF design**, under the stated independence assumption.

7. Telecom Billing – Candidate Keys, Primary Keys and Functional Dependencies

Representative Telecom Schema

Since the question does not specify the exact telecom attributes, consider:

BILLING(MobileNo, CustomerName, PlanID, PlanName, CallID, CallDate, Duration, Rate, Amount)

Assumptions

- MobileNo uniquely identifies a customer.
- PlanID uniquely identifies a plan.
- CallID uniquely identifies a call record.
- Each call belongs to one mobile number.
- Rate is determined by the selected plan.

Step 1: Functional Dependencies

1. **MobileNo $\rightarrow$ CustomerName, PlanID**
2. **PlanID $\rightarrow$ PlanName, Rate**
3. **CallID $\rightarrow$ MobileNo, CallDate, Duration**
4. **(CallID, MobileNo) $\rightarrow$ Amount**

Since CallID uniquely identifies the call, MobileNo is already determined by CallID.

Therefore:

CallID $\rightarrow$ MobileNo $\rightarrow$ CustomerName, PlanID

and:

PlanID $\rightarrow$ PlanName, Rate

Step 2: Determine Candidate Key

Calculate:

CallID+

Starting with:

{CallID}

Using CallID $\rightarrow$ MobileNo, CallDate, Duration:

{CallID, MobileNo, CallDate, Duration}

Using MobileNo $\rightarrow$ CustomerName, PlanID:

{CallID, MobileNo, CallDate, Duration, CustomerName, PlanID}

Using PlanID $\rightarrow$ PlanName, Rate:

{CallID, MobileNo, CallDate, Duration, CustomerName, PlanID, PlanName, Rate}

Therefore:

CallID+ = all attributes

Hence:

Candidate Key = CallID

Step 3: Primary Key

Since CallID is a candidate key and uniquely identifies each billing call:

Primary Key = CallID

Step 4: Non-Prime Attributes

All other attributes are non-prime attributes.

Final Answer

Candidate Key: CallID

Primary Key: CallID

Functional Dependencies:

- MobileNo $\rightarrow$ CustomerName, PlanID
- PlanID $\rightarrow$ PlanName, Rate
- CallID $\rightarrow$ MobileNo, CallDate, Duration
- CallID $\rightarrow$ Amount

Conclusion

The functional dependencies show that customer, plan, and call information should be separated during normalization to reduce redundancy and avoid anomalies.

8. Retail Inventory – Complete Normalization

Given Relation

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Assumptions

- PID uniquely identifies a product.
- SupplierID uniquely identifies a supplier.
- A product can be stored in multiple warehouses.
- A warehouse can store multiple products.

- Price is determined by the product.

Step 1: Functional Dependencies

1. **PID $\rightarrow$ PName, SupplierID, Category, Price**
2. **SupplierID $\rightarrow$ SupplierName**

There is a transitive dependency:

PID $\rightarrow$ SupplierID $\rightarrow$ SupplierName

There is also a many-to-many relationship between products and warehouses.

Step 2: Candidate Key

If a product can occur in multiple warehouses, the original relation requires:

(PID, Warehouse)

as the key.

Step 3: Partial Dependencies

The following dependencies exist:

PID $\rightarrow$ PName, SupplierID, Category, Price

These attributes depend only on PID, which is part of the composite key.

Therefore, partial dependencies exist.

Step 4: Decompose into 2NF

Create:

PRODUCT(PID, PName, SupplierID, Category, Price)

INVENTORY(PID, Warehouse)

Step 5: Remove Transitive Dependency

Since:

PID $\rightarrow$ SupplierID

and:

SupplierID $\rightarrow$ SupplierName

we create:

SUPPLIER(SupplierID, SupplierName)

Final Normalized Schema

PRODUCT(PID, PName, SupplierID, Category, Price)

Primary Key: **PID**

SUPPLIER(SupplierID, SupplierName)

Primary Key: **SupplierID**

INVENTORY(PID, Warehouse)

Primary Key: **(PID, Warehouse)**

Normalization Verification

- 1NF: All attributes contain atomic values.
- 2NF: Partial dependencies have been removed.
- 3NF: Transitive dependency $\text{SupplierID} \rightarrow \text{SupplierName}$ has been removed.
- BCNF: In each resulting relation, determinants are keys.

Conclusion

The retail inventory database is completely normalized up to **BCNF** under the stated assumptions. The decomposition reduces redundancy and supports multiple warehouses per product.

9. School ERP – Normalize up to BCNF

Representative Poorly Designed Relation

Since the question does not provide a specific schema, consider:

SCHOOL_ERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

Assumptions

- StudentID uniquely identifies a student.
- ClassID uniquely identifies a class.
- TeacherID uniquely identifies a teacher.
- SubjectID uniquely identifies a subject.
- A student can study multiple subjects.
- Grade depends on the student and subject.

Step 1: Functional Dependencies

1. **$\text{StudentID} \rightarrow \text{StudentName}, \text{ClassID}$**
2. **$\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$**

3. **TeacherID** $\rightarrow$ **TeacherName**
4. **SubjectID** $\rightarrow$ **SubjectName**
5. **(StudentID, SubjectID)** $\rightarrow$ **Grade**

Step 2: Candidate Key

The combination:

(StudentID, SubjectID)

determines all attributes.

Therefore:

Candidate Key = (StudentID, SubjectID)

Step 3: Identify Partial Dependencies

StudentID determines:

StudentName, ClassID

SubjectID determines:

SubjectName

These depend only on part of the composite key.

Therefore, the relation violates 2NF.

Step 4: Decompose into 2NF

Create:

STUDENT(StudentID, StudentName, ClassID)

SUBJECT(SubjectID, SubjectName)

RESULT(StudentID, SubjectID, Grade)

Step 5: Remove Transitive Dependencies

In STUDENT:

ClassID $\rightarrow$ **ClassName, TeacherID**

and:

TeacherID $\rightarrow$ **TeacherName**

Therefore, create:

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

Final BCNF Schema

STUDENT(StudentID, StudentName, ClassID)

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

SUBJECT(SubjectID, SubjectName)

RESULT(StudentID, SubjectID, Grade)

BCNF Verification

In STUDENT:

StudentID $\rightarrow$ StudentName, ClassID

StudentID is the key.

In CLASS:

ClassID $\rightarrow$ ClassName, TeacherID

ClassID is the key.

In TEACHER:

TeacherID $\rightarrow$ TeacherName

TeacherID is the key.

In SUBJECT:

SubjectID $\rightarrow$ SubjectName

SubjectID is the key.

In RESULT:

(StudentID, SubjectID) $\rightarrow$ Grade

The complete composite key determines Grade.

Therefore, all determinants are superkeys.

Conclusion

The School ERP schema is normalized up to **BCNF**. The decomposition removes partial and transitive dependencies, reduces redundancy, and prevents insertion, deletion, and update anomalies.

10. Movie Streaming Platform – Join Dependencies and 5NF

Representative Relation

Since the question does not specify the exact movie-streaming schema, consider:

MOVIE_STREAM(MovieID, ActorID, PlatformID)

Assumption

Assume the business rules state that:

- A movie can have multiple actors.
- A movie can be available on multiple streaming platforms.
- An actor can be associated with multiple platforms.
- These three relationships are independent and are represented by a valid join dependency.

Step 1: Identify the Join Dependency

The original relation combines three independent relationships:

1. Movie–Actor
2. Movie–Platform
3. Actor–Platform

Therefore, the relation can be represented as:

MOVIE_STREAM = MOVIE_ACTOR $\bowtie$ MOVIE_PLATFORM $\bowtie$ ACTOR_PLATFORM

This is a non-trivial join dependency.

Step 2: Identify Redundancy

Suppose a movie has several actors and is available on several platforms.

If all combinations are stored in one relation, the same movie-actor and movie-platform information may be repeated many times.

This produces unnecessary redundancy.

Step 3: 5NF Decomposition

Decompose the relation into:

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

Step 4: Join Dependency

The original relation can be reconstructed as:

MOVIE_STREAM = MOVIE_ACTOR $\bowtie$ MOVIE_PLATFORM $\bowtie$ ACTOR_PLATFORM

Therefore, the decomposition is based on the specified join dependency.

Step 5: Lossless-Join Verification

To verify the decomposition, join the three relations using their common attributes:

MOVIE_ACTOR $\bowtie$ MOVIE_PLATFORM $\bowtie$ ACTOR_PLATFORM

The original valid tuples are obtained when the stated business rule holds.

For example, suppose:

MOVIE_ACTOR

MovieID	ActorID
M1	A1
M1	A2

MOVIE_PLATFORM

MovieID	PlatformID
M1	P1
M1	P2

ACTOR_PLATFORM

ActorID	PlatformID
A1	P1
A2	P2

The join reconstructs the combinations permitted by the business rules.

Step 6: 5NF Verification

A relation is in 5NF, or Project-Join Normal Form, when every non-trivial join dependency is implied by candidate keys and no further lossless decomposition is required.

The decomposition separates the independent relationships and removes redundancy caused by the join dependency.

Conclusion

The movie-streaming relation is decomposed into:

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

The decomposition is **lossless under the stated join-dependency assumption** and represents the relation in **5NF**.